Monitoreo de Fauna Silvestre con Cámaras Trampa mediante YOLOv8: Generalización entre Ubicaciones y Reporte de Actividad Ecológica

Guillermo Cajo gcajos@uni.pe
Andre Carasas andre.carasas.o@uni.pe
Daniel Franco daniel.franco.s@uni.pe
Ivonne Lazo ivonne.lazo.g@uni.pe
David Grabiel david.grabiel.g@uni.pe
Joel Renojo joel.renojo.c@uni.pe

FIIS, Universidad Nacional de Ingeniería (UNI), Lima, Perú

Resumen

Las cámaras trampa generan más imágenes de las que un equipo humano puede revisar. Ajustamos un detector YOLOv8s sobre Caltech Camera Traps (CCT-20) para localizar y clasificar 15 categorías (14 especies y vehículos) y añadimos un módulo de monitoreo por reglas que resume la actividad por cámara, especie y periodo. Usamos la partición oficial *eccv_train*/*eccv_val* para separar ubicaciones vistas (*cis*) de ubicaciones nuevas (*trans*). El modelo, preentrenado en COCO y entrenado 50 épocas a 640 px con aumento fuerte (mosaic, mixup, copy-paste), alcanza un mAP@0.5 de 0.963 en validación y 0.901 en prueba *cis*, pero solo 0.484 y 0.563 en *trans*, lo que da brechas de 0.478 y 0.338. La brecha no es uniforme: especies como *rodent*, *squirrel*, *rabbit* y *coyote* pierden entre 0.48 y 0.90 de AP@0.5, mientras que *car*, *skunk* y *raccoon* casi no cambian. En ubicaciones nuevas el recall (IoU $\geq$ 0.5, conf $\geq$ 0.25) es 0.495 en animales pequeños y 0.831 en medianos. Grad-CAM da un enriquecimiento de 1.04 en ubicaciones vistas y 0.83 en nuevas, es decir, la atención no se concentra en el animal. Esto es compatible con dependencia del contexto, aunque no es una prueba concluyente. En la cámara 100 (no vista) el módulo de monitoreo detectó 11 especies y estimó una tasa de fotografías vacías de 0.086.

1 Motivación

Las cámaras trampa, activadas por movimiento y sin operador, son la herramienta más usada para monitorear biodiversidad en áreas protegidas, incluidas la Amazonía y los Andes peruanos. El cuello de botella es la revisión manual: gran parte de los disparos son falsas activaciones (viento, vegetación, cambios de luz) y el resto contiene animales parcialmente ocultos, pequeños o lejanos.

Desde el punto de vista del aprendizaje profundo, el problema es interesante por tres razones. Primero, el desbalance: en CCT-20 hay especies con miles de imágenes (*opossum*, *rabbit*) y otras con menos de diez (*fox*, *badger*). Segundo, la oclusión y el tamaño del animal, que hacen que muchas detecciones sean difíciles. Tercero, y lo más importante para este trabajo, la generalización geográfica: un modelo puede aprender el fondo característico de cada cámara (una roca, un tronco, un patrón de luz) en lugar de rasgos del animal, y su desempeño cae al desplegarse en una ubicación nueva. Por eso medimos esa brecha de forma explícita y buscamos evidencia de dependencia del fondo, además de traducir las detecciones en un reporte útil para un equipo de manejo de fauna.

2 Trabajos Relacionados

El trabajo más cercano es el de Beery et al. [1], que mostró que los modelos de reconocimiento en cámaras trampa generalizan mal a ubicaciones no vistas y publicó CCT-20 con particiones *cis* y *trans* para medir ese efecto; es el marco experimental de este informe. Norouzzadeh et al. [2] automatizaron la identificación de especies en Snapshot Serengeti con redes profundas, y Beery et al. [3] propusieron un esquema en dos etapas con un detector genérico de animales (MegaDetector) para descartar imágenes vacías antes de clasificar. Nuestro enfoque sigue la idea de detectar y clasificar en un solo paso con YOLO, pero evalúa de forma explícita la brecha entre ubicaciones.

Para la detección usamos la familia YOLO [4] y la implementación de Ultralytics [5] (YOLOv8); YOLOv9 [6] es la alternativa recomendada en el enunciado. Para el desbalance consideramos la pérdida focal [7] y el sobremuestreo de especies raras. Como aumentos que buscan debilitar la asociación entre fondo y especie usamos mixup [9] y copy-paste [10], junto con mosaic. Para interpretar el detector usamos Grad-CAM [8], que combina los gradientes de una clase con los mapas de una capa convolucional para resaltar las regiones que más influyen en la decisión.

3 Metodología

3.1 Datos y particiones

Usamos Caltech Camera Traps de LILA BC con las cajas del subconjunto ECCV18 (63 025 imágenes con anotación, 64 926 cajas y 85 ubicaciones) y el archivo de metadatos de imágenes, del que se toman las imágenes vacías. En el disco local se indexaron 244 584 imágenes .jpg y no faltó ninguna de las usadas. Siguiendo la partición oficial, las 10 ubicaciones de *eccv_train* forman el conjunto de entrenamiento y las 10 de *eccv_val* son las ubicaciones nuevas. Como el archivo no trae subconjuntos *cis*/*trans* de validación y prueba, los construimos así: (a) las 10 ubicaciones nuevas se reparten por ubicación en *val_trans* (5: 7, 28, 46, 105, 125) y *test_trans* (5: 0, 40, 78, 100, 130), con semilla 42; (b) de las ubicaciones de entrenamiento se reservan secuencias completas, no fotogramas sueltos, el 10 % para *val_cis* y otro 10 % para *test_cis*, de modo que una ráfaga casi idéntica no quede repartida entre entrenamiento y evaluación. El código verifica que ninguna ubicación *trans* aparezca en entrenamiento (0 ubicaciones compartidas) y que ninguna secuencia esté en dos particiones (0 repartidas). La brecha de generalización se define como la diferencia de mAP entre *cis* y *trans*. Una aclaración sobre el enunciado: el subconjunto con cajas contiene 15 categorías (14 especies y *car*), no 21, y el mapeo de clases se construye a partir de las anotaciones.

3.2 Reducción de escala y manejo de imágenes vacías

Las imágenes originales son pesadas, así que el script local las reduce a un lado mayor de 640 px, las exporta a formato YOLO y las comprime. Se exportaron 48 786 imágenes (con sus etiquetas) en 82 minutos con 4 hilos, y el archivo resultante pesa 3.95 GB (97 576 archivos). Esa carpeta comprimida se sube a Drive y el notebook de Colab la copia al disco local (3.68 GiB en 1.1 min), la descomprime (1.6 min) y verifica que los conteos de cada partición coincidan con la Tabla 2 (las cinco dicen OK). Las imágenes vacías se tratan como fondo puro: se exportan con archivo de etiquetas vacío y sin clase propia, para que el detector aprenda a no producir cajas. En entrenamiento se pidió una vacía por cada imagen con animal, pero en las 10 ubicaciones usadas solo hay 3 917 vacías en total (1 704 en entrenamiento), así que se usaron todas las disponibles.

Tabla 1: Decisiones de reducción de escala y de entrenamiento.

Aspecto	Decisión
Modelo	YOLOv8s preentrenado en COCO (11.1 M de parámetros, 28.5 GFLOPs)
Resolución	640 px (las imágenes ya vienen reducidas a ≤ 640 px de lado mayor)
Entrenamiento	22 487 imágenes (todo el *train*), lote 32, 50 épocas, semilla 42, determinista
Optimizador	Automático de Ultralytics (MuSGD, lr 0.01), decaimiento coseno, AMP
Aumento (*strong*)	mosaic 1.0, mixup 0.15, copy-paste 0.3, escala 0.6, HSV (0.03, 0.7, 0.5), rotación 5°, volteo 0.5
Mosaic	Activo en las épocas 1 a 40; se desactiva en las últimas 10 (valor por defecto)
Parada y selección	Paciencia 15 (no se activó); mejor punto de control según *val_cis*
Cómputo	GPU Tesla T4 en Colab; 4.9 h para las 50 épocas (≈ 5 a 6 min por época)

Tabla 2: Resumen del conjunto por partición.

Partición	Imágenes	Con animal	Vacías	Ubicaciones	Especies
Entrenamiento	22 487	20 783	1 704	10	15
Val. *cis*	2 775	2 555	220	10	14
Val. *trans*	10 638	9 473	1 165	5	12
Prueba *cis*	2 859	2 653	206	10	14
Prueba *trans*	10 027	9 405	622	5	14

Las particiones de evaluación tienen entre 6 % y 11 % de imágenes vacías, muy por debajo del ≈ 70 % del dataset completo; por eso los falsos positivos sobre imágenes vacías pesan poco en las métricas.

3.3 Justificación de las decisiones de entrenamiento

Las decisiones de la Tabla 1 se tomaron *a priori* y no se compararon con alternativas, por lo que sus justificaciones son técnicas y no empíricas.

Resolución de 640 px. Es la resolución con la que YOLOv8 se preentrena en COCO, así que el modelo ya está habituado a ese tamaño de entrada. También es un compromiso razonable entre la visibilidad de los animales pequeños y el costo de cómputo en una GPU T4. Reducir las imágenes a ese lado mayor bajó el conjunto a 3.95 GB, lo que permitió subirlo a Drive. No probamos otras resoluciones; el recall de 0.495 en animales pequeños (sección 4.3) sugiere que una resolución mayor podría ayudar.

Aumento fuerte. Mosaic combina fondos de varias imágenes en un mismo cuadro y mixup superpone dos imágenes; ambos apuntan al problema central del enunciado, que el modelo memorice el fondo de cada cámara. Copy-paste, que mueve animales a otros fondos, también estaba configurado (0.3), pero según el código fuente de Ultralytics 8.4.188 requiere máscaras de segmentación y nuestras etiquetas son solo cajas, por lo que no tuvo efecto en el entrenamiento.

Imágenes vacías como fondo. Es la práctica estándar en YOLO: una imagen sin etiquetas enseña al modelo a no producir falsas detecciones. Importa porque en un despliegue real la mayoría de los disparos están vacíos.

3.4 Métricas

Reportamos mAP@0.5 y mAP@0.5:0.95 con la validación de Ultralytics (umbrales por defecto) sobre cada partición, con desglose por especie. Para cuantificar el efecto del desbalance calculamos la correlación de Spearman entre el número de imágenes de entrenamiento de cada especie y su AP@0.5.

3.5 Robustez ante oclusión y tamaño

El conjunto no incluye anotación de oclusión, por lo que usamos un **indicador aproximado** (un contraste de la caja frente a un anillo de fondo, combinado con si la caja toca el borde de la imagen), que está pensado como proxy de camuflaje u oclusión parcial y no como medida directa. Un animal tiene oclusión *alta* si su caja toca el borde de la imagen o su contraste está en el tercil inferior, *media* si está en el tercil intermedio y *baja* si está en el superior. El tamaño se define por el área relativa

de la caja (pequeño < 1 %, mediano entre 1 % y 8 %, grande > 8 %). En cada grupo reportamos el recall sobre *test_trans*, contando como acierto una detección con IoU ≥ 0.5, misma clase y confianza ≥ 0.25.

3.6 Grad-CAM y módulo de monitoreo

Para Grad-CAM usamos la capa 9 del backbone (SPPF) y como objetivo la puntuación de clase máxima de la salida del detector. Medimos, en 100 imágenes con animal de *val_cis* y 100 de *val_trans*, la *energía en caja* (fracción del mapa de atención que cae dentro de las cajas reales) y el *enriquecimiento* (esa fracción dividida por la fracción de área que ocupan las cajas). Un valor cercano a 1 indica que la atención se reparte como el área, sin preferencia por el animal. Además revisamos seis casos cualitativos.

El módulo de monitoreo es un sistema de reglas que actúa sobre las detecciones de una cámara no vista. Aplica un umbral de confianza de 0.25, agrupa por especie y periodo (aquí, por día) y calcula conteo de detecciones, fotogramas con la especie, confianza media, primera y última aparición, y la tasa de fotografías vacías frente a las que contienen fauna. Un *evento* es una secuencia de disparos en la que la especie aparece en al menos 2 fotogramas, lo que evita contar varias veces al mismo animal.

4 Resultados

Todos los resultados corresponden a una única corrida con la configuración de la Tabla 1 (*B_strong*). Las configuraciones *A_base* (aumento suave) y *C_strong_os3* (sobremuestreo de especies raras) están previstas en el notebook, pero **no se ejecutaron**, por lo que su efecto no se midió.

4.1 Entrenamiento y rendimiento por partición

Las 50 épocas se completaron sin que se activara la parada temprana. El mAP@0.5 en *val_cis* pasa de 0.704 en la primera época a una meseta de 0.96 a partir de la época 33 aproximadamente, y el mejor punto de control es de la época 39 según el registro. La pérdida de clasificación de entrenamiento cae de golpe en la época 41 (de ≈ 0.69 a ≈ 0.45), justo cuando se apaga el mosaic, lo que indica que las imágenes sin mosaic son más fáciles para el modelo (Figura 1).

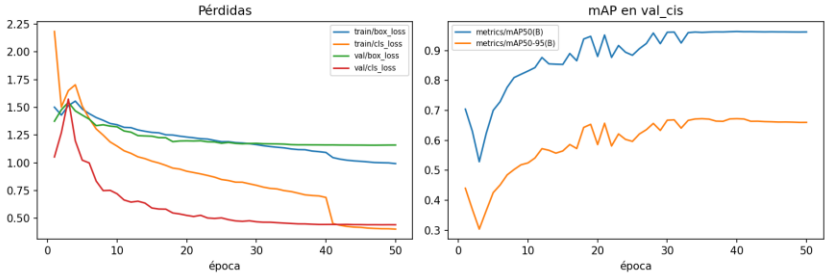


Figura 1: Pérdidas y mAP en *val_cis* por época.

Tabla 3: Métricas por partición con el punto de control elegido en *val_cis* (P y R son precisión y recall medios).

Partición	mAP@0.5	mAP@0.5:0.95	P	R
Val. *cis*	0.963	0.665	0.889	0.952
Val. *trans*	0.484	0.307	0.719	0.454
Prueba *cis*	0.901	0.635	0.929	0.876
Prueba *trans*	0.563	0.384	0.734	0.511
Brecha en validación (*cis* − *trans*)	+0.478	+0.358		
Brecha en prueba (*cis* − *trans*)	+0.338	+0.251		

La validación *cis* no es independiente: se usó para elegir el punto de control, por lo que su cifra es optimista. La brecha en prueba es la más fiable.

En ubicaciones vistas el detector funciona muy bien (mAP@0.5 de 0.901 en prueba), pero en ubicaciones nuevas cae a 0.563: la brecha es de 0.338 en mAP@0.5 y de 0.251 en mAP@0.5:0.95,

y tiene el mismo signo y un tamaño parecido en validación (0.478). Para valorar cuánto de esto es azar, las dos particiones *trans* difieren entre sí en 0.079 (0.484 frente a 0.563), cuatro veces menos que la brecha en prueba; con solo 5 ubicaciones por partición esto no basta para dar un intervalo, pero indica que la brecha es mucho mayor que la variación entre conjuntos de ubicaciones nuevas. El tiempo de inferencia fue de unos 4 a 8 ms por imagen en la T4.

4.2 Desglose por especie y desbalance de clases

Tabla 4: AP@0.5 por especie en prueba y número de imágenes de entrenamiento. "—": no hay AP para esa clase en esa partición.

Especie	Imágenes en train	AP@0.5 *cis*	AP@0.5 *trans*	Brecha
fox	4	0.495	0.000	0.495
badger	7	—	0.463	—
deer	138	0.966	—	—
skunk	295	0.939	0.936	0.003
rodent	354	0.916	0.020	0.896
bird	742	0.619	0.263	0.356
dog	1 130	0.927	0.788	0.139
bobcat	1 223	0.971	0.802	0.169
car	1 231	0.995	0.994	0.001
squirrel	1 334	0.945	0.395	0.550
raccoon	1 498	0.979	0.899	0.080
coyote	2 028	0.971	0.496	0.475
cat	2 255	0.953	0.516	0.437
rabbit	3 169	0.953	0.418	0.535
opossum	5 375	0.988	0.887	0.101

La columna de brecha (*cis* − *trans*) es un cálculo propio a partir de las dos columnas anteriores.

La brecha global esconde diferencias grandes entre especies. Sobre las 13 especies con AP en ambas pruebas, el AP@0.5 medio baja de 0.896 a 0.570. *Car* (0.995 frente a 0.994), *skunk* (0.939 frente a 0.936), *raccoon* y *opossum* casi no pierden, mientras que *rodent* (de 0.916 a 0.020), *squirrel*, *rabbit*, *coyote* y *cat* pierden entre 0.44 y 0.90. Llama la atención que *rabbit* y *cat* tienen miles de imágenes de entrenamiento y aun así pierden más de 0.43, mientras que *skunk* (295 imágenes) no pierde nada. Por lo tanto, la brecha no se explica solo por la frecuencia de la especie: apunta a efectos de apariencia y de ubicación. Las especies con pocos ejemplos (*fox*, *badger*) tienen AP bajo o muy variable, pero con 1 a 8 instancias estos valores son anecdóticos.

La correlación de Spearman entre la frecuencia en entrenamiento y el AP@0.5 es de **0.591** en *cis* (n = 14) y de **0.314** en *trans* (n = 14, p ≈ 0.27, cálculo propio a partir de la Tabla 4). La frecuencia se asocia con el AP en ubicaciones vistas, pero casi nada en ubicaciones nuevas, donde pesan más otros factores.

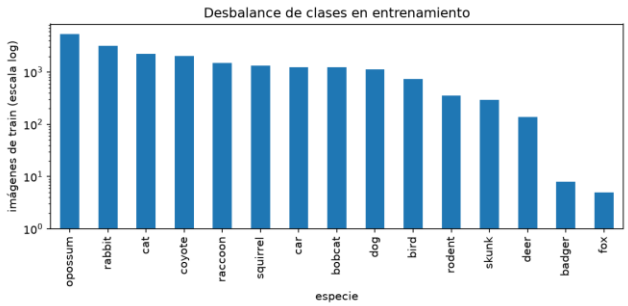


Figura 2: Imágenes de entrenamiento por especie (escala logarítmica). Nueve especies tienen menos de 1 300 ejemplos, y *fox* y *badger*, menos de 10.

No evaluamos ninguna estrategia de mitigación del desbalance: la configuración con sobremuestreo (*C_strong_os3*) no se ejecutó. Lo único que mitiga el desbalance en nuestra corrida es el aumento *strong* (mosaic, mixup, copy-paste), sin una comparación contra una línea base.

4.3 Robustez ante tamaño y oclusión

Tabla 5: Recall (IoU ≥ 0.5, conf ≥ 0.25) por grupo en *test_trans*.

Factor	Grupo	n (cajas)	Recall
Oclusión (indicador aprox.)	bajo	2 267	0.775
Oclusión (indicador aprox.)	medio	2 917	0.841
Oclusión (indicador aprox.)	alto	4 664	0.758
Tamaño relativo	grande (> 8 %)	2 107	0.736
Tamaño relativo	mediano (1–8 %)	7 032	0.831
Tamaño relativo	pequeño (< 1 %)	709	0.495

El tamaño es el factor dominante: el recall cae de 0.831 en objetos medianos a 0.495 en pequeños (−0.336), es decir, se pierde la mitad de los animales pequeños. Los objetos grandes (0.736) rinden peor que los medianos, algo que atribuimos a que suelen estar cortados por el borde de la imagen, aunque no lo comprobamos. El indicador de oclusión varía poco entre grupos (como máximo 0.083) y no es monótono: el nivel *bajo* (0.775) rinde peor que el *medio* (0.841). Por eso no lo tomamos como evidencia de que la oclusión afecte poco al detector; más bien, el proxy no la mide de forma fiable. Con la sola información de la Tabla 5 estimamos un recall global de ≈ 0.79 en ubicaciones nuevas (cálculo propio, ponderando por n).

4.4 Interpretabilidad con Grad-CAM

Tabla 6: Grad-CAM: fracción del mapa dentro de las cajas reales (100 imágenes por partición).

Partición	Energía en caja	Fracción de área	Enriquecimiento
Ubicaciones vistas (*val_cis*)	0.066	0.066	1.040
Ubicaciones nuevas (*val_trans*)	0.054	0.071	0.833

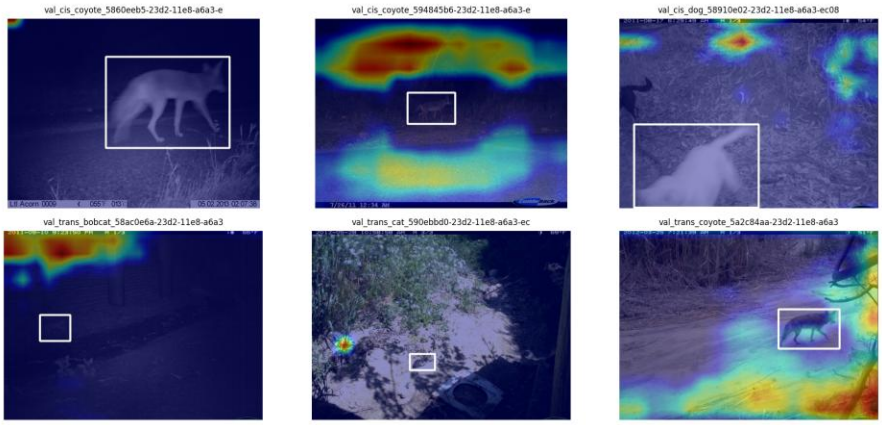


Figura 3: Grad-CAM (azul = baja, rojo = alta activación). Arriba, ubicaciones vistas (coyote, coyote, perro); abajo, ubicaciones nuevas (lince, gato, coyote). El cuadro blanco es la caja real.

En ubicaciones vistas la atención cae sobre el animal tanto como le corresponde por área (enriquecimiento de 1.04), y en ubicaciones nuevas cae menos de lo que le correspondería (0.83): la energía en caja baja de 0.066 a 0.054 aunque las cajas ocupan una fracción de área parecida (0.066 y 0.071). Los casos cualitativos de la Figura 3 son coherentes con esto: en casi todos el mapa se activa sobre el fondo (la franja superior de la imagen, el suelo o la vegetación) y no sobre el animal. En el coyote de arriba a la izquierda casi no hay activación, y en el gato de abajo el único punto de calor cae lejos de la caja. Solo en el coyote de abajo a la derecha parte de la activación roza el animal.

Esto es compatible con que el modelo use el contexto de la escena, pero no lo demuestra. Hay una tensión que no logramos resolver: en ubicaciones vistas el detector llega a 0.90 de mAP@0.5 y aun

así su mapa de atención no prefiere al animal, lo que sugiere que Grad-CAM sobre este backbone no refleja bien lo que usa la cabeza de detección. Además, en ubicaciones nuevas el modelo falla más, y esos fallos pueden contribuir al menor enriquecimiento (no separamos imágenes con y sin detección correcta). Tomamos estos resultados como una señal, no como una prueba.

4.5 Módulo de monitoreo ecológico

Aplicamos el módulo a la cámara 100, que pertenece a *test_trans* (no se vio en entrenamiento) y es la más poblada: 4 320 imágenes tomadas entre mayo de 2010 y junio de 2011. Con confianza ≥ 0.25 se detectó fauna en 3 948 fotogramas y no se detectó nada en 372, lo que da una **tasa estimada de vacías de 0.086**; se observaron 11 especies distintas. Esa tasa es una estimación basada en las predicciones, no en la verdad de campo, por lo que hereda los errores del modelo. La Tabla 7 (apéndice) detalla la actividad por especie.

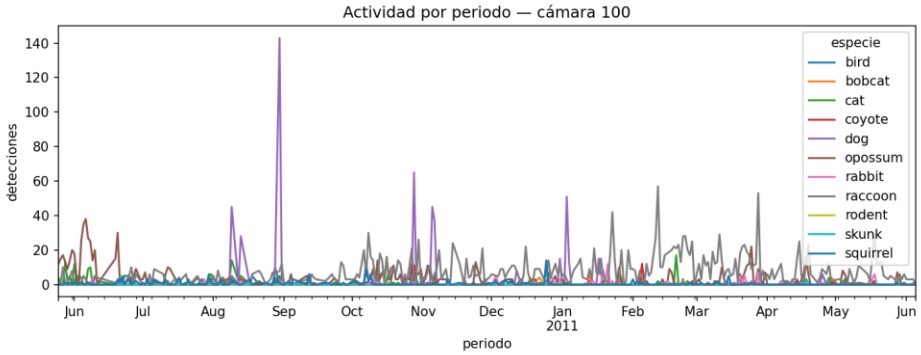


Figura 4: Detecciones diarias por especie en la cámara 100.

La cámara está dominada por el mapache (2 342 de 4 471 detecciones, 52 %), seguido de la zarigüeya (910) y el perro (493). Las especies con menor confianza media (*skunk*, *bird*, *rodent*, *rabbit*) son las que un equipo de manejo debería revisar a mano; *rodent* y *skunk* suman un solo evento cada una. El pico de la Figura 4 a fines de agosto (más de 140 detecciones de perro en un día) merece verificación manual, porque puede deberse a una ráfaga real o a falsos positivos repetidos.

5 Discusión: generalización y uso responsable de la IA en conservación

Generalización y fondo. La brecha entre ubicaciones vistas y nuevas es grande y consistente: 0.478 en validación y 0.338 en prueba, mucho mayor que la diferencia entre las dos particiones *trans* (0.079). No es uniforme: las especies muy distintivas o de alto contraste (*car*, *skunk*, *raccoon*, *opossum*) generalizan casi sin pérdida, y las pequeñas, crípticas o de aspecto variable (*rodent*, *squirrel*, *rabbit*, *coyote*, *cat*) pierden mucho, incluso si tienen miles de ejemplos de entrenamiento. Grad-CAM apunta en la misma dirección (enriquecimiento de 0.83 en ubicaciones nuevas y mapas sobre el fondo), pero con las limitaciones de la sección 4.4, así que no podemos afirmar que el modelo memorice el fondo de cada cámara; solo que su desempeño no se transfiere de forma pareja y que hay indicios de dependencia del contexto.

Costo de errores. Un falso negativo puede excluir de las decisiones de manejo a una especie amenazada que sí estuvo presente, y su costo es acumulativo y difícil de detectar después. Un falso positivo cuesta el tiempo de un especialista que revisa una detección incorrecta, un costo acotado y visible. Nuestros resultados muestran dónde se concentran los falsos negativos: en ubicaciones nuevas, con confianza 0.25 se pierde cerca del 21 % de los animales (recall global de ≈ 0.79) y la mitad de los pequeños, y las especies con menor AP en ubicaciones nuevas (*fox* 0.000, *rodent* 0.020, *bird* 0.263) suelen ser las menos frecuentes. Por eso conviene operar con umbrales que favorezcan el recall para las especies prioritarias y mantener revisión humana de las detecciones de baja confianza, como las de la Tabla 7.

Despliegue sin validación local. Desplegar un modelo entrenado en una región en otra, por ejemplo uno entrenado con cámaras del suroeste de EE. UU. en la selva amazónica peruana, sin validarlo

localmente es riesgoso: la fauna, la vegetación y las cámaras son distintas. Si dentro de un mismo dataset ya observamos que el mAP@0.5 baja 0.34 al pasar a cámaras nuevas, es razonable esperar una brecha al menos igual entre regiones; no la medimos aquí. Se recomienda validar con una muestra anotada localmente, reportar métricas por cámara y reentrenar o ajustar con datos de la zona antes de usar el sistema para decisiones de manejo.

6 Conclusiones

6.1 Principales hallazgos

- **Un detector ligero basta en ubicaciones conocidas.** YOLOv8s preentrenado, entrenado 50 épocas, alcanza 0.901 de mAP@0.5 en prueba *cis*.
- **La generalización a cámaras nuevas es el problema.** El mAP@0.5 baja a 0.563 en *trans* (brecha de 0.338 en prueba y 0.478 en validación), y la pérdida se concentra en especies pequeñas o de aspecto variable, no solo en las raras.
- **El tamaño pesa más que la oclusión.** El recall de los animales pequeños es 0.495 frente a 0.831 en medianos; el indicador de oclusión no mostró un efecto claro.
- **La interpretabilidad deja una señal, no una prueba.** El enriquecimiento de Grad-CAM de 1.04 y 0.83 indica que la atención no se concentra en el animal, compatible con dependencia del contexto.
- **El módulo de monitoreo funciona de extremo a extremo** sobre una cámara no vista (4 320 imágenes, 11 especies, tasa de vacías estimada de 0.086), aunque sus cifras heredan los errores del detector.

6.2 Limitaciones

- Solo se ejecutó una configuración (*B_strong*) y una semilla: no hay comparación entre aumentos *base* y *strong*, ni evaluación del sobremuestreo, ni estimación de la variación entre corridas.
- *val_cis* se usó para elegir el punto de control, por lo que sus cifras son optimistas.
- Las particiones *trans* tienen 5 ubicaciones cada una y sus especies difieren; el mAP no es directamente comparable entre ellas, y las especies raras tienen muy pocas instancias (*fox*, *badger*) y métricas inestables.
- El subconjunto tiene solo 6 a 11 % de imágenes vacías, por lo que la penalización de falsos positivos sobre vacías está subestimada respecto de un despliegue real.
- El indicador de oclusión es aproximado y no resultó fiable; no separa oclusión de camuflaje, iluminación o tamaño.
- Grad-CAM se evaluó en 100 imágenes por partición y con seis casos cualitativos; no separamos los casos con y sin detección correcta, y el mapa del backbone puede no reflejar la decisión de la cabeza de detección.
- La tasa de vacías del monitoreo se estima con predicciones y no se contrastó con verdad de campo; la regla de eventos (≥ 2 fotogramas) no distingue individuos.
- Las imágenes se redujeron a un lado mayor de 640 px antes de entrenar, lo que puede perjudicar a los animales pequeños; el mAP se calcula con Ultralytics y puede diferir levemente del protocolo oficial.

6.3 Trabajo futuro

- Ejecutar las configuraciones pendientes (*A_base* y *C_strong_os3*) para medir el efecto del aumento y del sobremuestreo, y repetir con varias semillas.
- Evaluar la brecha por ubicación individual en vez de por grupos de 5 ubicaciones, y añadir un segundo dataset de LILA BC de otra región.
- Separar las imágenes con y sin detección correcta en el análisis de Grad-CAM y usar mapas de mayor resolución.
- Probar un filtro previo con un detector genérico de animales (MegaDetector [3]) y técnicas de generalización de dominio.

- Mejorar la medición de oclusión con anotaciones manuales de una muestra.

El código, las instrucciones de reproducción y la declaración de uso de IA están en el Apéndice B.

Referencias

[1] Beery, S., Van Horn, G., & Perona, P. (2018). Recognition in Terra Incognita. ECCV. arXiv:1807.04975.

[2] Norouzzadeh, M. S., et al. (2018). Automatically identifying, counting, and describing wild animals in camera-trap images with deep learning. PNAS, 115(25).

[3] Beery, S., Morris, D., & Yang, S. (2019). Efficient Pipeline for Camera Trap Image Review. arXiv:1907.06772.

[4] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR.

[5] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics.

[6] Wang, C.-Y., Yeh, I.-H., & Liao, H.-Y. M. (2024). YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. arXiv:2402.13616.

[7] Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal Loss for Dense Object Detection. ICCV.

[8] Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV.

[9] Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mixup: Beyond Empirical Risk Minimization. ICLR.

[10] Ghiasi, G., et al. (2021). Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation. CVPR.

A Tablas y figuras complementarias

Este apéndice reúne la tabla de detalle del módulo de monitoreo citada en el texto principal.

Tabla 7: Actividad por especie en la cámara 100, con confianza ≥ 0.25. Eventos: secuencias con la especie en ≥ 2 fotogramas.

Especie	Detecciones	Fotogramas	Eventos	Conf. media	Primera	Última
raccoon	2 342	2 200	759	0.781	2010-05-25	2011-06-04
opossum	910	906	258	0.734	2010-05-25	2011-05-31
dog	493	366	124	0.791	2010-08-09	2011-05-05
cat	272	268	64	0.674	2010-05-25	2011-05-18
squirrel	138	136	36	0.647	2010-05-28	2011-05-28
rabbit	125	116	28	0.556	2010-06-07	2011-06-04
bird	70	59	17	0.488	2010-06-02	2011-04-29
bobcat	68	68	14	0.600	2010-07-06	2011-06-05
coyote	45	44	9	0.629	2010-08-13	2011-05-04
rodent	5	5	1	0.523	2010-10-16	2011-02-23
skunk	3	3	1	0.459	2011-04-18	2011-04-19

B Material complementario y reproducibilidad

El código reproducible se encuentra en la carpeta *Pregunta03* del repositorio de GitHub: **https://github.com/dfranco79/redes_neuronales_examen_parcial_grupo05**. El flujo se ejecuta en dos partes. La parte local (*preparar_local.py*) lee el JSON de CCT-20, indexa las imágenes, arma las particiones *cis*/*trans*, verifica que no haya fuga de ubicaciones ni de secuencias, exporta las imágenes reducidas a formato YOLO y genera *yolo_data_cct20.zip*; una segunda ejecución con los datos ya exportados tarda 2.6 min. La parte de Colab se ejecuta desde el notebook *cct20_colab.ipynb*, que monta Drive, copia y verifica los datos, entrena (o reutiliza o reanuda una corrida previa si ya existe), evalúa, mide la robustez, calcula Grad-CAM y genera el reporte de monitoreo. La lógica está en los módulos *colab_comun.py*, *entrenar.py*, *evaluar.py*, *robustez.py*, *gradcam_cct.py* y *monitoreo.py*; el notebook solo los orquesta. Para cambiar de configuración basta editar *EXPERIMENTO* (*A_base*, *B_strong* o *C_strong_os3*) en la celda de configuración. Entorno de Colab: GPU Tesla T4, Python 3.13, PyTorch 2.11 y Ultralytics 8.4.188. Los pesos, tablas y figuras de la corrida *yolov8s_B_strong* quedan en la carpeta de resultados de Drive.

Declaración de uso de IA. Usamos un asistente de IA para apoyar la redacción y el formato de este informe y para revisar y ajustar partes del código de los notebooks. Las decisiones de diseño, la ejecución de los experimentos y el análisis de resultados fueron revisados por el grupo.